

Project: Smart Parking Platform

Version: 4.2

Date: 10.1.2026

Table of Contents

1. **Introduction**
 - 1.1 Purpose
 - 1.2 Scope
2. **Overall Description**
 - 2.1 Users
 - 2.2 System Environment
 - 2.3 Constraints
 - 2.4 Assumptions
3. **Functional Requirements**
4. **Non-Functional Requirements**
5. **Use Cases**
6. **Work Breakdown Structure (WBS)**
7. **Project Timeline and Schedule**
 - 7.1 Story Point Estimates and Risks
 - 7.2 Project Schedule
 - 7.3 Gantt Chart
 - 7.4 Milestones
8. **Agile Development Plan**
 - 8.1 Product Backlog
 - 8.2 Sprint Planning
 - 8.3 Trello Board
9. **Risk Management**
 - 9.1 Risk Categories
 - 9.2 Risk Assessment
 - 9.3 Risk Response Strategies
 - 9.4 Risk Register
10. **Quality Management**
 - 10.1 Quality Assurance
 - 10.2 Quality Control
 - 10.3 Quality Metrics
11. **Communication Management Plan**
 - 11.1 Meeting Cadence
 - 11.2 Reporting Methods
 - 11.3 Communication Tools

Software Requirements Specification (SRS)

1. Introduction

1.1 Purpose

This document defines the initial requirements for the Smart Parking Platform developed by WE ARE Software Corp. The system will help drivers locate, reserve, and pay for parking spaces in real time while providing parking operators with tools to manage their facilities and monitor occupancy. The platform will be available through web and mobile applications and will provide parking availability information, navigation assistance, and management tools for parking operators. The goal is to develop a project management plan for the creation of this system, including requirements gathering, stakeholder management, scheduling, budgeting, risk management, and team coordination.

1.2 Scope

The project will provide web and mobile applications that support parking searches, real-time availability, reservations, digital payments, navigation, notifications, and parking management. The project will also provide an administrative dashboard for parking operators to manage facilities and view occupancy information and reports. Physical parking garage construction, manufacturing parking hardware, and direct control of city traffic systems are out of scope.

2. Overall Description

2.1 Users:

- Vehicle Owners
- Parking Facility Owners/Employees
- Finance and Billing Employees
- Third-Party Payment Providers
- City Transportation Employees
- Mobile Phone Users

2.2 System Environment:

The Smart Parking Platform will consist of web-based and mobile applications connected to backend/API services and a database. The system will also integrate with external mapping/navigation and payment services.

2.3 Constraints:

- The project has a fixed semester timeline.
- The project has an unlimited theoretical development budget.
- The system depends on third-party mapping, navigation, payment, and other APIs.
- The system must follow applicable security and privacy requirements.
- The system must consider applicable city ordinances and laws.
- Users must authenticate to access protected account and management functions.

2.4 Assumptions:

- Users have internet access.
- Users have a compatible web browser or mobile device.
- Parking facilities provide accurate availability and occupancy information.
- Third-party mapping and payment services are available when required.
- Parking operators will provide and maintain accurate facility information.

3. Functional Requirements

FR1: The system shall allow users to register and log in securely.

FR2: The system shall allow users to search for available parking locations.

FR3: The system shall display parking locations and availability on an interactive map.

FR4: The system shall allow users to reserve available parking spaces.

FR5: The system shall process digital parking payments through a supported payment provider.

FR6: The system shall provide users with reservation history and receipts.

FR7: The system shall send users reservation-related notifications and alerts.

FR8: The system shall provide navigation assistance to selected parking facilities through an external mapping service.

FR9: The system shall allow parking operators to add and manage parking facilities.

FR10: The system shall allow parking operators to monitor parking occupancy.

FR11: The system shall provide occupancy and parking usage reports.

FR12: The system shall allow authorized users to manage relevant account and parking information.

Additional functional requirements will be identified through stakeholder analysis and requirements elicitation throughout the semester.

4. Non-Functional Requirements

NFR1: The system shall support at least 1,000 concurrent users.

NFR2: The system shall display parking availability updates within 1 second when current data is available.

NFR3: The system shall target 99.99% uptime for critical services.

NFR4: The system shall protect user account, vehicle, reservation, and payment-related information.

NFR5: The system shall support modern web browsers and supported mobile devices.

NFR6: The system shall be scalable to support additional users and parking facilities.

NFR7: The system shall maintain consistent reservation and payment information when transactions are completed successfully.

5. Use Cases

Use Case 1

Title: Reserve a Parking Space

Actor: Driver

Precondition: User is logged in and a parking facility has an available space.

Steps:

- User searches for a parking location.

- User selects an available parking facility.
- User selects the reservation date and time.
- System verifies parking availability.
- User confirms the reservation and completes payment.
- System displays the reservation confirmation.

Postcondition: The parking reservation is saved to the user's account.

Work Breakdown Structure (WBS)

1. Smart Parking Platform

1.1 User & Account Management

1.1.1 User Registration & Login

Account creation

Secure login

1.1.2 User Profile

Account information

Vehicle information

1.1.3 Authentication & Security

Authentication controls

Protected user data

1.2 Parking Search & Availability

1.2.1 Parking Location Search

Search available facilities

Filter parking locations

1.2.2 Real-Time Availability

Display available spaces

Update occupancy information

1.2.3 Interactive Parking Map

Display parking locations

Select parking facility

1.3 Reservations

1.3.1 Space Selection

Select parking facility

Select available space

1.3.2 Reservation Management

Select date and time

Confirm reservation

1.3.3 Reservation History

View past reservations

View reservation details

1.4 Payments

1.4.1 Payment Processing

- Connect payment provider

- Process parking payment

1.4.2 Payment Confirmation

- Confirm successful payment

- Store transaction information

1.4.3 Receipts

- Generate digital receipt

- View receipt history

1.5 Navigation & Notifications

1.5.1 Navigation Integration

- Connect mapping service

- Provide directions to facility

1.5.2 User Notifications

- Reservation confirmation

- Reservation alerts

1.6 Parking Operator Management

1.6.1 Facility Management

- Add parking facility

- Update facility information

- Remove facility

1.6.2 Occupancy Management

- Monitor occupancy

- Update availability

1.6.3 Operator Dashboard

- View facility information

- View parking activity

1.7 Reporting & Analytics

1.7.1 Occupancy Reports

- Generate occupancy reports

- View occupancy trends

1.7.2 Parking Usage Reports

- Analyze parking usage

- Review reservation activity

1.8 Testing & Deployment

1.8.1 System Testing

- Functional testing

- Integration testing

1.8.2 Security & Performance Testing

- Security testing

- Performance testing
- 1.8.3 Deployment
 - Deploy web application
 - Deploy mobile application
 - Deploy backend services

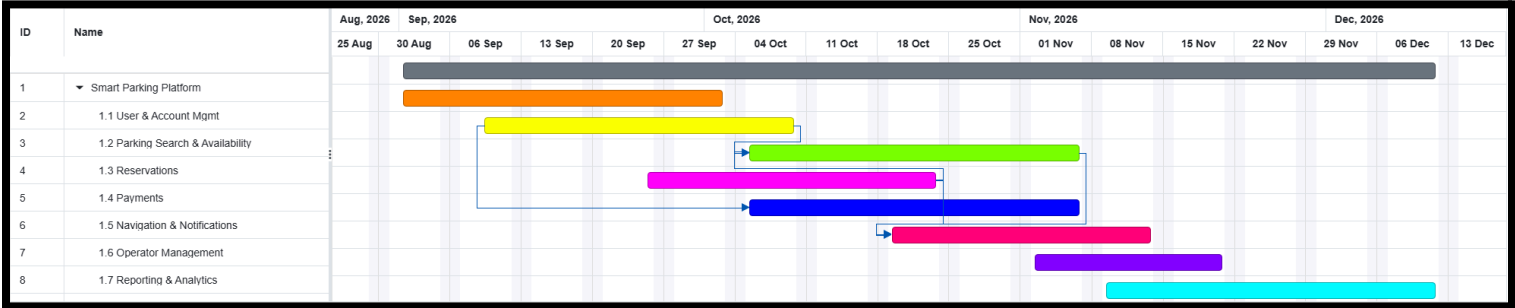
Project Timeline

WBS Component	Story Points	Risk
1.1 User & Account Mgmt	8	Moderate complexity; relies on secure auth standards
1.2 Parking Search & Availability	5	Depends on third-party map API reliability
1.3 Reservations	13	High complexity; requires state management & concurrency handling
1.4 Payments	8	Standard gateway integration but needs error/retry logic
1.5 Navigation & Notifications	3	Lightweight API calls; mostly configuration-driven
1.6 Operator Management	5	CRUD-heavy but straightforward UI/backend mapping
1.7 Reporting & Analytics	8	Requires data aggregation & charting libraries
1.8 Testing & Deployment	5	Cross-team coordination; environment setup overhead

WBS Component	Start	End	Duration	Dependencies
1.1 User & Account Mgmt	Sept Week 1	Oct Week 1	~5 wks	–
1.2 Parking Search & Availability	Sept Week 2	Oct Week 2	~5 wks	–
1.3 Reservations	Oct Week 1	Nov Week 1	~5 wks	1.2 (Finished)
1.4 Payments	Sept Week 4	Oct Week 3	~5 wks	1.4 (Started)
1.5 Navigation & Notifications	Oct Week 2	Nov Week 1	~5 wks	1.3 (Started)
1.6 Operator Management	Oct Week 3	Nov Week 2	~4 wks	1.3 (Finished~), 1.4 (Started)

1.7 Reporting & Analytics	Nov Week 1	Nov Week 3	~3 wks	Backend API Calling
1.8 Testing & Deployment	Nov Week 2	Dec Week 2	~5 wks	First working Dev component required to start testing

Gantt Chart



Milestones

Milestone	Target Date	Success Criteria
Requirements & WBS Baseline	Sept 7 (Week 1)	SRS v1.0 approved, WBS signed off by team leads
Sprint 1: Core Search & Auth Complete	Oct 9 (Week 6)	Users can log in, search lots, view live availability
Sprint 2: Reservation & Payment Flow Live	Nov 6 (Week 10)	End-to-end booking + payment confirmation working
Sprint 3: Operator Dashboard & Reports Ready	Nov 27 (Week 13)	Facilities manageable, occupancy/reports viewable
Sprint 4: Final Testing and Product Release	Dec 11th (Week 15)	Product Testing Completed, Product Released to public and team reorganized to focus on maintenance

Product Backlog

Login

1	User Registration	Sprint 1
2	User Login	Sprint 1
3	Secure Password Handling	Sprint 1
4	User Logout	Sprint 1
5	Role-based Access for Users and Operators	Sprint 1
6	Authentication Testing	Sprint 1
7	Password Reset	Sprint 2

User/Operator UI

1	User Registration Screen	Sprint 1
2	User Login Screen	Sprint 1
3	User Dashboard	Sprint 1
4	User Profile Screen	Sprint 1
5	Operator Dashboard	Sprint 1
6	Vehicle Information Screen	Sprint 2
7	Parking Search Screen	Sprint 2
8	Interactive Parking Map	Sprint 2
9	Parking Availability Display	Sprint 2
10	Parking Facility Details	Sprint 2
11	Operator Facility Management Screen	Sprint 2
12	Parking Reservation Screen	Sprint 3
13	Payment Screen	Sprint 3

14	Reservation Confirmation Screen	Sprint 3
15	Reservation Confirmation Notification	Sprint 3
16	Upcoming Reservation Reminder	Sprint 4
17	Reservation Cancellation Notification	Sprint 4
18	Reservation History Screen	Sprint 4
19	Payment History Screen	Sprint 4
20	Digital Receipt History Screen	Sprint 4

User/Operator Backend

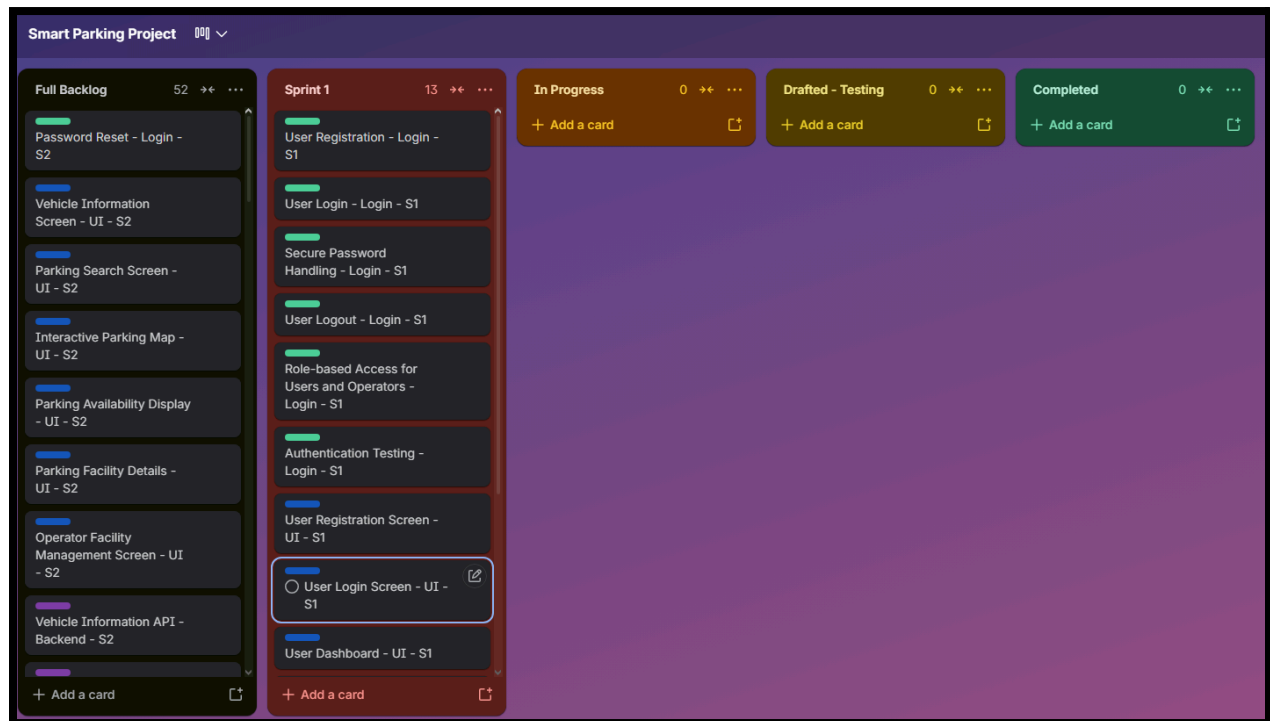
1	User Account API	Sprint 1
2	Authentication API	Sprint 1
3	User Profile API	Sprint 1
4	Vehicle Information API	Sprint 2
5	Parking Facility API	Sprint 2
6	Parking Search API	Sprint 2
7	Real-time Availability API	Sprint 2
8	Occupancy Update API	Sprint 2
9	Operator Facility Management API	Sprint 2
10	Update Parking Facility Information	Sprint 2
11	Remove Parking Facility	Sprint 2
12	Update Parking Space Availability	Sprint 2
13	Integrate External Mapping Service	Sprint 2
14	Secure API Access and Authorization	Sprint 2
15	Reservation API	Sprint 3
16	Reservation Availability Validation	Sprint 3
17	Reservation Cancellation API	Sprint 3

18	Payment Processing Integration	Sprint 3
19	Payment Confirmation Processing	Sprint 3
20	Prevent Double Booking of Parking Spaces	Sprint 3
21	Handle Failed Payment Transactions	Sprint 3
22	Operator Occupancy Monitoring	Sprint 3
23	Provide Navigation to Selected Parking Facility	Sprint 3
24	Receipt Generation Service	Sprint 4
25	Reservation Cancellation and Refund Processing	Sprint 4
26	Record account and System Activity Logs	Sprint 4
27	Payment Integration Testing	Sprint 4
28	Final System Integration Testing	Sprint 5

Reporting Frontend/Backend

1	Reservation History	Sprint 3
2	Payment History	Sprint 3
3	Occupancy Report	Sprint 4
4	Daily Occupancy Report	Sprint 4
5	Weekly Occupancy Report	Sprint 4
6	Parking Usage Report	Sprint 4
7	Reservation Activity Report	Sprint 4
8	Facility Utilization Report	Sprint 5
9	Revenue/Payment Report	Sprint 5
10	Performance Testing for Concurrent Users	Sprint 5

Trello Board



[https://trello.com/invite/b/6ab5f184a1a2cb8fdbc54f2f/ATTI5ea9c23d87646c071dbde30ad65b95320CEA08EE/smart-parking-pr
oject](https://trello.com/invite/b/6ab5f184a1a2cb8fdbc54f2f/ATTI5ea9c23d87646c071dbde30ad65b95320CEA08EE/smart-parking-project)

Risk Management

Risk management is important to the Smart Parking Platform because the project depends on real-time parking information, third-party services, multiple development teams, and a fixed semester timeline. Risks will be identified, assessed, monitored, and addressed throughout the project.

Technical Risks

ID	Risk Description
T1	Third-party mapping or navigation service becomes unavailable.

T2	Payment processing service experiences an outage or integration problem.
T3	Real-time parking availability information is inaccurate or delayed.
T4	Web and mobile applications have compatibility problems.

Schedule Risks

ID	Risk Description
S1	Requirements changes increase development time.
S2	Reservation functionality takes longer than planned.
S3	Third-party API dependencies delay development.
S4	Testing identifies defects that require additional development time.

Financial Risks

ID	Risk Description
F1	Third-party service fees change during the project.
F2	Payment provider fees are higher than expected.
F3	Additional security or compliance requirements create costs.
F4	Additional software or service subscriptions are needed.

- The project has an unlimited theoretical budget, but these risks are still tracked because outside services and project decisions may affect resource planning.

People Risks

ID	Risk Description
P1	A team member becomes unavailable during development.
P2	A development team lacks experience with a required technology or service.
P3	Communication problems occur between development teams.
P4	Stakeholder availability limits requirements feedback or approvals.

Risk Assessment

Probability Scale

Value	Probability
5	Very High (>80%)
4	High (61–80%)
3	Moderate (41–60%)
2	Low (21–40%)
1	Very Low (<20%)

Impact Scale

Value	Impact
5	Catastrophic
4	Major

3	Moderate
2	Minor
1	Negligible

Risk Matrix

Probability \ Impact	1	2	3	4	5
5 Very High	Medium	High	High	Extreme	Extreme
4 High	Low	Medium	High	High	Extreme
3 Moderate	Low	Medium	Medium	High	High
2 Low	Low	Low	Medium	Medium	High
1 Very Low	Low	Low	Low	Medium	Medium

Risk Response Strategies

- Avoid: Change the project plan to eliminate a risk when possible.
- Mitigate: Reduce the likelihood or impact through actions such as testing, early integration, and backup plans.
- Transfer: Shift part of a risk to a third-party provider, such as a payment or mapping service.
- Accept: Monitor a risk when its impact is limited or prevention is not practical.

Risk Register

ID	Risk	Category	Probability	Impact	Level	Owner	Response
----	------	----------	-------------	--------	-------	-------	----------

R1	Mapping service outage	Technical	4	4	High	Mapping Team	Mitigate
R2	Payment service failure	Technical	3	5	High	Payment Team	Mitigate
R3	Inaccurate parking availability	Technical	4	4	High	Backend Team	Mitigate
R4	Web/mobile compatibility issues	Technical	3	3	Medium	Mobile/ Web Teams	Mitigate
R5	Requirements changes delay development	Schedule	3	4	High	Project Manager	Mitigate
R6	Reservation development delay	Schedule	4	4	High	Backend Team	Mitigate
R7	Third-party API delays	Schedule	3	4	High	Project Manager	Mitigate
R8	Testing requires major rework	Schedule	3	4	High	QA Team	Mitigate

R9	Third-party service fees increase	Financial	2	3	Medium	Project Manager	Accept
R10	Payment provider fees increase	Financial	2	3	Medium	Finance Team	Accept
R11	Additional security/compliance costs	Financial	2	4	Medium	Security Team	Mitigate
R12	Additional software subscriptions required	Financial	2	2	Low	Project Manager	Accept
R13	Team member becomes unavailable	People	3	4	High	Project Manager	Mitigate
R14	Technology skill gap	People	3	3	Medium	Team Lead	Mitigate
R15	Poor communication between teams	People	4	4	High	Project Manager	Mitigate
R16	Limited stakeholder availability	People	3	4	High	Project Manager	Mitigate

Quality Management

Quality Assurance

- Requirements reviews
- Design reviews
- Coding standards
- Peer code reviews
- Security reviews
- Sprint retrospectives

Quality Control

- Unit testing
- Integration testing
- System testing
- User acceptance testing
- Security testing
- Performance testing
- Regression testing
- Web and mobile compatibility testing

Quality Metrics

Metric	Target
System Availability	99.99%
Concurrent Users	1,000+
Availability Refresh	Less than 1 second

Critical Defects Before Release	0
Payment Processing Success	99%+
Reservation Accuracy	99%+

Communication Management Plan

Communication will coordinate the Mobile Application, Web Application, Backend/API, Mapping & Location Services, Payment Integration, and Quality Assurance teams while keeping project stakeholders informed.

Meeting Cadence

Meeting	Frequency	Participants	Purpose
Daily Stand-Up	Daily	Development teams, Project Manager	Progress and blockers
Team Review	Weekly	Team leads, Project Manager	Progress, risks, and dependencies
Sprint Review	Every 2 weeks	Project team, stakeholders	Demonstrate completed work
Stakeholder Meeting	Monthly	Project stakeholders	Project status and major decisions

Reporting Methods

Audience	Method	Frequency
Development Teams	Trello updates	Daily

Team Leads	Status report	Weekly
Project Manager	Risk register review	Weekly
Stakeholders	Sprint demonstrations	Bi-Weekly
Parking Operators	Progress presentation	Monthly
Management	Project dashboard/KPI report	Monthly

Communication Tools

- Trello: Backlog, sprint planning, and task tracking
- Microsoft Teams: Team communication and meetings
- Email: Formal stakeholder communication
- Sprint Reviews: Project demonstrations
- Risk Register: Risk tracking
- Status Reports: Project monitoring